



04 | State Is Just A Variable: Reactivity In Svelte Five

*The problem-and-answer review companion to the audio lecture of the same name.
Read the question, try to answer it, then check the short answer, the map, and the files.
Quiz yourself with the index first, and drill each card with its quick-fire review table.*

Legend

-  `Q{N}` — a technical question. Numbered so you can jump ("Q11").
- ⚡ Short answer — the direct verdict (Yes/No or the key move) plus a one-line technical resolution, before the teaching.
- 🧠 Theory — the full explanation, built from the basics up.
- Component map — a quick sketch of the files and how they relate.
 Symbols:  a component (`.svelte`),  a module (`.svelte.ts`),  a point inside a component; indentation = nested-inside or used-by;  its role; /  props/events.
- 📄 The files — the real code, each shown in full, named with `title=" ... "`.
- 🔑 The move — the one-line habit to carry away.
- ⚠ Gotcha — a trap or surprise.
- 🍷 Going further — an edge case or a curiosity. The dessert.
- 🔄 Quick-fire review — a small table of clear question-and-answer pairs that drill the same card; cover the right column and test yourself.
- ♦ Source — where the code comes from (a docs file, or reconstructed / for comparison). Printed at the foot of each entry.

Sources: `/documentation 2026 June/svelte-docs/02-runes/02-$state.md`, plus framing from `/documentation 2026 June/claude-discussions`. Files marked *reconstructed* are minimal components built around the docs' examples. Real-world use-case framing for `$state.raw` and `svelte/reactivity` (Q6–Q7) drawn from the Svelte docs and tutorial: [svelte.dev/docs/svelte/\\$state](https://svelte.dev/docs/svelte/$state) [[https://svelte.dev/docs/svelte/\\$state](https://svelte.dev/docs/svelte/$state)], svelte.dev/docs/svelte/svelte-reactivity [<https://svelte.dev/docs/svelte/svelte-reactivity>], svelte.dev/tutorial/svelte/raw-state [<https://svelte.dev/tutorial/svelte/raw-state>].

• • •

The index (cover the answers, test yourself)

Making a value react to change

- Q1 — Reactivity basics: a plain `let claps = 0` changes in memory but the screen never updates. How do I make a variable reactive?

When state is an object or a list

- Q2 — Deep reactivity: with a `$state` list, is a nested field like `tasks[0].done` reactive too — and does `tasks.push( ... )` update the screen?
- Q3 — Destructuring: does `let { title } = article` give me a `title` that stays in sync with `article.title`?

Modeling data as a class

- Q4 — Reactive class fields: in a `Draft` class, do the fields become reactive automatically — and if not, how?
- Q5 — Method binding: does `onclick={draft.reset}` correctly run `reset` on the draft?

Reactive built-ins and the variants

- Q6 — Reactive built-ins: a `Set` of liked post IDs won't update the UI on `add`. How do I make a `Set` / `Map` / `Date` / `URL` reactive?
- Q7 — Performance at scale: a feed of thousands of posts I only ever replace or append to — how do I skip the cost of deep-proxying?
- Q8 — Handling state outside Svelte: why does `structuredClone(article)` throw on a `$state` object, and how do I get a plain copy?
- Q9 — Synchronous feedback: a `$state` change can lag a beat behind a click. How do I force the UI to reflect it instantly?

Passing state around

- Q10 — Passing state to a function: I call a function with a state value

`(greet(name))`. Does it get a live link to `name`, or just a copy of its current value?

Sharing state across files

- Q11 — Sharing across files: where do I put one value — the logged-in reader — that many components read and update together?
- Q12 — Sharing, encapsulated: how do I let components read the logged-in reader but route every change through one controlled place?

Going further

- Q13 — One file, two components: can I define both `ArticleCard` and `ClapButton` in a single `.svelte` file?
- Q14 — `$state` vs `$derived` vs `$effect`: which one holds the "5 min read" label computed from the article body?
- Q15 — Framework bridge: is `$state` just React's `useState` or Vue's `ref` under a new name?

• • •

Making a value react to change

Q1 — Reactivity basics: a plain `let claps = 0` changes in memory but the screen never updates. How do I make a variable reactive?

⚡ **Short answer.** Declare it with `$state`: `let claps = $state(0)`. That marks the variable for the compiler, which wires it to the screen, so every change (`claps++`) repaints the spots that show it. No setter, no `.value`.

🧠 **Theory.** Start with a plain variable, say a number called `claps`. In ordinary JavaScript, changing that number does nothing to the screen — the variable lives in memory, and the page has no idea it changed. To update what the reader sees, you'd have to find the right element yourself and rewrite its text by hand, every single time. `$state` removes that chore. Wrapping the starting value in `$state` makes the variable *reactive*, which just means Svelte now watches it: whenever you change `claps`, every spot on screen that shows it redraws itself, automatically. And you change it like any normal number, `claps++`, with no special setter and no ceremony.

Component map.

- ArticleCard.svelte → owns claps (\$state)

The files.

ArticleCard.svelte

```
<script>
  let claps = $state(0);
</script>

<article>
  <h2>Ceasefire Talks Collapse</h2>
  <button onclick={() => claps++}>clap ({claps})</button>
</article>
```

🔑 **The move.** Declare it with `$state`, then change it like any plain variable. No setter, no `.value`.

🌱 **Going further.** `$state` is *deep*: hand it an object or an array and every level is reactive too. That's Q2.

🔄 **Quick-fire review.**

QUESTION	ANSWER
What does a plain <code>let count = 0</code> do on screen when you change it?	Nothing — it changes in memory, but the page has no idea, so you'd have to rewrite the DOM by hand.
How do you make a variable reactive?	Declare it with the <code>\$state</code> rune: <code>let count = \$state(0)</code> .
What does "reactive" actually mean here?	Svelte watches the variable, so whenever you change it, every place on screen that shows it redraws automatically.
Do you need a special setter to change it?	No — you change it like any normal variable (<code>count++</code>), with no setter and no <code>.value</code> .

◆ Source: `02-runes/02-$state.md` (the counter, shown as a full component)

• • •

When state is an object or a list

Q2 — Deep reactivity: with a `$state` list, is a nested field like `tasks[0].done` reactive too — and does `tasks.push( ... )` update the screen?

⚡ **Short answer. Yes** to both. `$state` wraps an object or array in a proxy *deeply* — every level, all the way down — and the proxy even notices `push`. So `tasks[0].done = !tasks[0].done` and `tasks.push( ... )` each update exactly the right slice of the screen.

🧠 **Theory.** A clap count is a single number, but real state is often richer: an object (a bundle of labelled values, like an article with a title and a body) or a list of them. So what happens when you hand one of those to `$state`? Svelte wraps it in a *proxy* — picture a proxy as a watchful layer that sits in front of your object and quietly notices every time you read or write one of its fields. And it does this *deeply*: not only the outer object, but every object and array nested inside it, all the way down. That's what makes it feel effortless. Reach in and flip one field (`tasks[i].done = ...`), or `push` a new item onto the list, and the proxy sees exactly what changed and updates only that slice of the screen. (If you remember React, this is where `push` normally betrays you: mutating state instead of replacing it leaves the screen frozen. Here the proxy is watching, so the plain, obvious code just works.)

Component map.

- `NewsroomChecklist.svelte` → owns `tasks` (`$state` — a deep proxy, item by item)

📁 The files.

📄 `NewsroomChecklist.svelte`

```
<script>
  let tasks = $state([ { done: false, text: 'add the photo' } ]);
</script>

{#each tasks as task, i}
  <label>
    <input type="checkbox" checked={task.done}
      onchange={() => tasks[i].done = !tasks[i].done} />
    {task.text}
  </label>
{/each}

<button onclick={() => tasks.push({ done: false, text: 'legal check' })}>
  add task
</button>
```

🔑 **The move.** Mutate nested state the plain way — assign, `push`, splice. The proxy is watching.

⚠️ **Gotcha.** Updating a proxy does not mutate the original underlying object. You'll never notice day to day; it only matters if you bring your own proxy handlers (wrap the object first, then `$state` it).

🔄 **Quick-fire review.**

QUESTION	ANSWER
If a <code>\$state</code> value is an object or array, are its nested fields reactive too?	Yes — <code>\$state</code> makes it <i>deeply</i> reactive, wrapping every level, so changing a nested field updates the screen.
Does calling <code>tasks.push(...)</code> on a <code>\$state</code> array update the UI?	Yes — the proxy notices the <code>push</code> (and other mutations), so the list re-renders.
How is that different from React?	In React, mutating state with <code>push</code> is a bug that freezes the screen; in Svelte the proxy is watching, so the plain mutation just works.
Does updating the proxy change the original underlying object?	No — the proxy records the change in its own layer; it only matters if you bring your own proxy handlers.

◆ **Source:** `02-runes/02-$state.md` (the to-do list, toggle, and push)

• • •

Q3 — Destructuring: does `let { title } = article` give me a `title` that stays in sync with `article.title`?

⚡ **Short answer. No.** Destructuring copies the value out once, right at that line. The copy has no link back to the proxy, so later changes to `article.title` never reach it. Read `article.title` directly when you need it to stay reactive.

🧠 **Theory.** First, recall what you're holding. You declared the article as reactive state — a variable Svelte watches so the screen follows it when it changes. And this variable isn't a single value; it's an *object*, a bundle of labelled fields like `title` and `claps`. As we saw in Q2, Svelte makes a reactive object work by wrapping it in a proxy, a watchful layer that notices every read and write to those fields.

Now, *destructuring*. That's an everyday JavaScript shorthand for lifting a field out of an object into its own standalone variable: instead of `let title = article.title`, you write `let { title } = article`, and JavaScript pulls `title` out for you. Convenient — and here's the trap. The instant you destructure, JavaScript reads the value out and *copies* it into your new variable, once, right then. That copy is a loose, plain string with no wire back to the proxy. So later, when you change `article.title`, the proxy faithfully updates everything still reading *through* it, but your copied `title` was never connected to the proxy, so it can't hear the change. It stays frozen at whatever the title was the moment you destructured. (This isn't a Svelte quirk; it's just how copying works in JavaScript, and Vue's reactivity stumbles on exactly the same move.)

Component map.

- `ArticleCard.svelte` → owns `article` (`$state`)
 - `let { title } = article` → a dead copy, frozen at this line

📁 The files.

ArticleCard.svelte

```

<script>
  let article = $state({ title: 'Ceasefire Talks Collapse', claps: 0
});

  // ✖ Trap: a copy taken now, with no live link
  let { title } = article;

  function rename() {
    article.title = 'Ceasefire Holds';
  }
</script>

<h2>{title}</h2>           <!-- stays "Ceasefire Talks Collapse" -->
<h2>{article.title}</h2>   <!-- updates to "Ceasefire Holds" -->
<button onclick={rename}>rename</button>

```

🔑 **The move.** Don't destructure what must stay reactive. Reach in fresh:

`article.title`.

🔄 **Quick-fire review.**

QUESTION	ANSWER
If you write <code>let { title } = article</code> , does <code>title</code> update when <code>article.title</code> later changes?	No — it stays frozen at the value it had the moment you destructured it.
Why does destructuring break the reactive link?	It copies the value out into a new variable once, at that line; the copy has no connection back to the reactive proxy, so it never hears later changes.
How do you keep the value reactive instead?	Don't pull the field into its own variable — read it directly each time, as <code>article.title</code> , so you always go through the proxy.
Is this a Svelte-specific quirk?	No — it's how value copying works in plain JavaScript; Vue's reactivity has the exact same pitfall.

◆ **Source:** `02-runes/02-$state.md` (destructuring a reactive value)

• • •

Modeling data as a class

Q4 — Reactive class fields: in a `Draft` class, do the fields become reactive automatically — and if not, how?

⚡ **Short answer. No, not automatically** — class instances aren't proxied. You mark each reactive field yourself with `$state`: on the field (`done = $state(false)`), or on its first assignment in the constructor (`this.text = $state(text)`).

🧠 **Theory.** Sometimes, instead of a loose object, you reach for a *class*. A class is a blueprint for making objects: you describe the shape once — the fields it holds, and the actions (called *methods*) it can perform — then stamp out as many objects from it as you like. Each object you create from the blueprint is an *instance*, and you make one with the word `new`, like `new Draft('headline')`. Why bother? Because a class keeps a thing's data and the functions that act on that data together in one tidy package — a natural fit for a draft article that knows how to reset or publish itself.

Here's the wrinkle for reactivity. When you hand `$state` a plain object, Svelte automatically wraps it in that watchful proxy layer (the one from Q2) so it becomes reactive. But it deliberately does *not* do that to a class instance. Why hold back? A class is a fussier creature than a plain object — it has methods, sometimes private fields, its own internal machinery — and clamping a proxy around the whole instance could quietly break that machinery. So Svelte leaves classes alone and asks you to switch on reactivity yourself, one field at a time: write `$state` directly on the field (`done = $state(false)`), or on its first assignment inside the *constructor* (the constructor is the setup function that runs once, automatically, the moment you create the instance). Either spot makes that single field reactive.

Component map.

- `NewsroomEditor.svelte` → `new Draft(...)` — reads `draft.text`, toggles `draft.done`
 - ↳ uses ▸ `draft.svelte.ts` → `class Draft` — each reactive field marked `$state`

📁 **The files.**

draft.svelte.ts

```
export class Draft {
  done = $state(false);

  constructor(text) {
    this.text = $state(text);
  }

  reset() {
    this.text = '';
    this.done = false;
  }
}
```

NewsroomEditor.svelte

```
<script>
  import { Draft } from './draft.svelte.ts';
  let draft = new Draft('Breaking: ceasefire talks resume');
</script>

<h3>{draft.text}</h3>
<button onclick={() => draft.done = !draft.done}>
  {draft.done ? 'ready to publish' : 'mark ready'}
</button>
```

🔑 **The move.** In a class, mark each reactive field yourself: `done = $state(false)`, or `this.text = $state(text)` in the constructor.

⚠️ **Gotcha.** The compiler turns those fields into hidden get/set methods, so they're non-enumerable — loop over the instance's keys and they won't show up. Rare, but real.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Is a plain object handed to <code>\$state</code> reactive? Is a class instance?	A plain object is wrapped and made reactive automatically; a class instance is not.
Why doesn't Svelte auto-wrap a class instance?	A class has methods and maybe private fields; wrapping the whole instance in a proxy could break that machinery, so Svelte leaves it alone.
How do you make a class field reactive, then?	Put <code>\$state</code> on the field (<code>done = \$state(false)</code>), or on its first assignment in the constructor (<code>this.text = \$state(text)</code>).
What is an "instance," and how do you make one?	An object stamped out from the class blueprint, created with <code>new</code> — e.g. <code>new Draft('headline')</code> .

◆ **Source:** `02-runes/02-$state.md` (the `Todo` class, renamed `Draft` for our newsroom)

• • •

Q5 — Method binding: does `onclick={draft.reset}` correctly run `reset` on the draft?

⚡ **Short answer. No.** Passing the method bare detaches it, so when the click fires, `this` inside `reset` is the `<button>`, not the draft, and the wrong object gets cleared. Use `onclick={() => draft.reset()}`, or define `reset` as an arrow-function field.

🧠 **Theory.** This one is pure JavaScript, not reactivity, but it bites constantly. Recall that our class has a *method*, a function that lives on the object, like `reset`. You'd like a button to call it, so you write `onclick={draft.reset}`, handing the button the method. Reasonable — and it doesn't work, because of the word `this`.

Inside a method, `this` means "the object the method was called on." The cruel detail is that `this` is decided by *how* the function is called, not where it was written. When you hand `draft.reset` over bare, you've torn the function loose from the draft. Later, when the click fires, the browser calls that loose function as the button's own handler — so inside `reset`, `this` is now the `<button>`, not the draft. The method dutifully clears fields, but on the wrong object, and your draft never resets. The two fixes both keep `this` pointed at the draft. Either call it *through* the draft with a tiny inline arrow (`() => draft.reset()`, so the `draft.` is right there when it runs), or define `reset` as an arrow function on the class — arrow functions don't get their own `this`, they permanently borrow it from where they were created (the instance), so it stays glued to the draft no matter who calls it.

Component map.

- `NewsroomEditor.svelte` → the `<button>` calls `draft.reset`
 - ↳ uses ▸ `draft.svelte.ts` → class `Draft` — inside `reset()`, `this` depends on HOW it's called`

The files.

 `NewsroomEditor.svelte`

```

<!-- ✖ Trap: `this` becomes the <button> -->
<button onclick={draft.reset}>discard</button>

<!-- ✔ Fix one: call it through the draft -->
<button onclick={() => draft.reset()}>discard</button>

```

draft.svelte.ts

```

export class Draft {
  done = $state(false);

  constructor(text) {
    this.text = $state(text);
  }

  // ✓ Fix two: an arrow field - `this` is bound to the instance for
  good
  reset = () => {
    this.text = '';
    this.done = false;
  };
}

```

🔑 **The move.** Pass `() => draft.reset()`, or define the method as an arrow-function field.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Does <code>onclick={draft.reset}</code> correctly reset the draft when clicked?	No — passing the method bare detaches it, so <code>this</code> inside <code>reset</code> becomes the <code><button></code> , not the draft.
What decides what <code>this</code> points to inside a method?	How the function is called, not where it was written; handed over bare, it loses its link to the object.
What's the first fix?	Call it through the draft with an inline arrow: <code>onclick={() => draft.reset()}</code> .
What's the second fix?	Define <code>reset</code> as an arrow-function field on the class, so its <code>this</code> is permanently bound to the instance.

◆ **Source:** `02-runes/02-$state.md` (the `this` traps and their two fixes)

• • •

Reactive built-ins and the variants

Q6 — Reactive built-ins: a `Set` of liked post IDs won't update the UI on `add`. How do I make a `Set` / `Map` / `Date` / `URL` reactive?

⚡ **Short answer. Use the reactive twin.** A plain `Set` / `Map` / `Date` / `URL` won't react under `$state`. Import `SvelteSet` (or `SvelteMap`, `SvelteDate`, `SvelteURL`) from `svelte/reactivity` and use it exactly like the original — now `add`, `delete`, `has`, and `size` all drive the UI.

🧠 **Theory.** First, why a `Set` here at all. On a social feed you need to know, for every post on screen, whether the current reader has liked it, so each post's heart can fill in. You could keep an array of liked IDs, but then "is this one liked?" scans the whole array on every render. A `Set` is the right structure: it holds unique values (you can't like twice) and answers `liked.has(id)` instantly. This membership shape is everywhere in real apps — the selected rows in a table, the expanded comment threads, the users currently online.

Now the catch. We saw in Q2 that `$state` makes a plain object or array reactive by wrapping it in a proxy. But a `Set`, `Map`, `Date`, or `URL` isn't a plain object — each keeps its contents in its own internal machinery that a proxy can't see through. So put a plain `Set` in `$state`, call `.add(id)`, and nothing on screen moves. Svelte's answer is a small package, `svelte/reactivity`, with drop-in reactive versions: `SvelteSet`, `SvelteMap`, `SvelteDate`, `SvelteURL` (and `SvelteURLSearchParams`). Same methods as the built-ins you know, but reading them — `has`, `size`, iterating — inside your markup registers a dependency, so an `add` or `delete` re-renders exactly the hearts that changed. Swap `new Set()` for `new SvelteSet()` and you're done.

Component map.

```

▸ likes.svelte.ts → owns liked (SvelteSet of post IDs) – shared across the feed
  └─ imported by ▣ PostCard.svelte → reads liked.has(post.id) → fills the heart;
    toggles on click

```

The files.

 likes.svelte.ts

```

import { SvelteSet } from 'svelte/reactivity';

export const liked = new SvelteSet(); // post IDs the reader has liked

export function toggleLike(id) {
  liked.has(id) ? liked.delete(id) : liked.add(id);
}

```

PostCard.svelte

```
<script>
  import { liked, toggleLike } from './likes.svelte.ts';
  let { post } = $props();
</script>

<button onclick={() => toggleLike(post.id)}>
  {liked.has(post.id) ? '♥' : '♡'} {post.text}
</button>
```

🔑 **The move.** Need a reactive `Set` / `Map` / `Date` / `URL`? Import its twin from `svelte/reactivity` — a plain one won't react.

🔧 **Going further.** The rest, by the job they actually do: `SvelteMap` for an id→value lookup that updates (a cache of fetched posts keyed by id, or per-post like *counts*); `SvelteDate` for a live clock or a "2 min ago" timestamp that ticks; `SvelteURL` / `SvelteURLSearchParams` for binding feed filters to the address bar, so the URL and the UI stay in lockstep.

🔄 **Quick-fire review.**

QUESTION	ANSWER
If you put a <code>Set</code> in <code>\$state</code> and call <code>.add()</code> , does the screen update?	No — a plain <code>Set</code> inside <code>\$state</code> is not reactive, so adding or removing items changes nothing on screen.
Why does <code>\$state</code> make plain objects and arrays reactive, but not a <code>Set</code> ?	<code>\$state</code> wraps the value in a proxy that watches its fields; a <code>Set</code> keeps its items in internal storage the proxy can't see into, so it never notices the change.
What's the fix?	Import the ready-made reactive version — <code>SvelteSet</code> , <code>SvelteMap</code> , <code>SvelteDate</code> , or <code>SvelteURL</code> — from <code>svelte/reactivity</code> and use it just like the built-in.
When would you choose a <code>SvelteMap</code> over a plain object?	When you want a reactive key-to-value lookup that updates the UI, such as a cache of items keyed by id, or a like-count per post.

◆ **Source:** `02-runes/02-$state.md` (built-in classes) + the `svelte/reactivity` docs; scenario reconstructed (an X-style feed) — see top Sources

• • •

Q7 — Performance at scale: a feed of thousands of posts I only ever replace or append to — how do I skip the cost of deep-proxying?

⚡ **Short answer.** Use `$state.raw`. It holds the value without deep-proxying it, so thousands of post objects cost nothing to wrap. The rule: you can't mutate it (no `posts.push( ... )`, no `posts[0].liked = ...`) — you update it only by *reassigning* a whole new array, which is exactly the immutable append an infinite feed already does.

🧠 **Theory.** By default `$state` is *deep*: it wraps your value and every object and array nested inside it in a proxy, so any change, however deep, is tracked (Q2). For a handful of items that's free and wonderful. But picture an infinite feed — an X-style timeline, a YouTube watch list — that grows to thousands of post objects, each with an author, counts, media. Wrapping every one of those in a proxy just so the list can render is real wasted work; at that scale the proxy overhead starts to bite. And you don't even need it: you never reach in and edit post number 847 in place. You load a page and *append* a new array; you switch feeds and *replace* the array.

That's the exact shape `$state.raw` is built for. It stores the value as-is, with no deep proxy, so the cost disappears. In return it listens for one kind of change only: reassignment. "Mutate" means reach inside — `posts.push(p)` or `posts[0].liked = true` — and raw ignores that completely. "Reassign" means hand the variable a brand-new value — `posts = [ ... posts, ... nextPage ]` — and that it tracks. Since the idiomatic way to grow a feed is already to build a new array (`[ ... posts, ... next ]`) rather than `push`, raw fits like a glove. Per-post state that *does* flip, like which posts are liked, lives outside the raw array — in a `SvelteSet` of IDs (Q6).

One honest caveat the Svelte team stresses: don't reach for `$state.raw` "for performance" by reflex. Proxies are fast enough for the vast majority of state. Profile first; reach for raw when you've actually got thousands of items, or a value you only ever swap.

Component map.

- `Feed.svelte` → owns `posts` (`$state.raw` — big array, only ever reassigned)
 - `loadMore()`: `posts = [ ... posts, ... next ]` → append = reassign → tracked
 - `posts.push( ... )` / `posts[0].liked = ...` → mutation → ignored ×
 - ↳ ■ `PostCard.svelte` → gets post (prop)

📁 The files.

Feed.svelte

```
<script>
  import PostCard from './PostCard.svelte';

  let posts = $state.raw([]); // thousands of posts; never edited
  in place

  async function loadMore() {
    const next = await fetchNextPage();
    posts = [...posts, ...next]; // ✓ reassign a new array –
    tracked
  }
</script>

{#each posts as post (post.id)}
  <PostCard {post} />
{/each}

<button onclick={loadMore}>load more</button>
```

🔑 **The move.** `$state.raw` for big arrays you only ever swap or append. Reassign a new value; never mutate inside it. And profile before reaching for it.

🔧 **Going further.** Same tool, other real cases: a large API response you replace on each fetch, or a big dataset you hand to a charting / visualization library that doesn't need Svelte tracking its insides. Raw can still *contain* reactive state (a raw array of ordinary reactive objects), and it works on class fields too.

🔄 **Quick-fire review.**

QUESTION	ANSWER
What does <code>\$state.raw</code> do?	It holds a value without deep-proxying it, skipping the cost of wrapping every nested item.
What's the one rule of raw state?	You can't mutate it (no <code>push</code> , no field edits) — you update it only by reassigning a whole new value.
Why does <code>posts = [... posts, ... next]</code> work but <code>posts.push(...)</code> doesn't?	Raw state listens only for reassignment; building and assigning a new array is a reassignment, while <code>push</code> is an in-place mutation it ignores.
When should you actually reach for it?	For large data you only ever swap or append — thousands of items, big API responses — and only after profiling, since proxies are fast enough for most state.

◆ **Source:** `02-runes/02-$state.md` (`$state.raw`); scenario reconstructed (an infinite social / video feed) — see top Sources

• • •

Q8 — Handling state outside Svelte: why does `structuredClone(article)` throw on a `$state` object, and how do I get a plain copy?

⚡ **Short answer.** It throws because reactive state is a `Proxy`, which outside tools like `structuredClone` don't accept. Wrap it in `$state.snapshot(article)` to get a plain, static copy with the proxy stripped off.

🧠 **Theory.** Remember that a reactive object isn't really your object anymore — it's a proxy, that watchful wrapper Svelte put around it. Inside Svelte, that's exactly what you want. The trouble starts when you hand your data to code that lives *outside* Svelte: a charting library, the browser's built-in `structuredClone` (which deep-copies data), even a plain `console.log`. Those tools weren't built to understand Svelte's proxy, so they get confused, print something cryptic, or flatly refuse it. `$state.snapshot` solves this: pass it your reactive value and it hands back a plain, ordinary, static copy of the data with the proxy peeled away — a frozen photograph, safe to give to anyone.

Component map.

- `ArticleCard.svelte` → owns `article` (`$state`, a `Proxy`)
- `$state.snapshot(article)` → a plain `{ ... }` for the outside world

📁 The files.

📄 `ArticleCard.svelte`

```
<script>
  let article = $state({ slug: 'ceasefire-talks', claps: 0 });

  function exportIt() {
    // logs `{ slug, claps }`, not `Proxy { ... }`
    console.log($state.snapshot(article));

    // structuredClone(article) would throw on the proxy –
    snapshot first:
    const copy = structuredClone($state.snapshot(article));
  }
</script>

<button onclick={exportIt}>export</button>
```

🔑 **The move.** Outside tool confused by the proxy? Wrap it in `$state.snapshot` to get a plain copy.

⚠ **Gotcha.** If the value has a `toJSON` method, the snapshot clones what `toJSON` returns instead of the raw object.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Why does <code>structuredClone(article)</code> throw when <code>article</code> is <code>\$state</code> ?	Because reactive state is a <code>Proxy</code> , and tools outside Svelte like <code>structuredClone</code> don't accept proxies.
How do you get a plain, non-proxy copy of reactive state?	Wrap it in <code>\$state.snapshot(article)</code> , which returns a plain static copy with the proxy stripped off.
When do you need a snapshot?	Whenever you hand state to code outside Svelte — a clone, a charting library, even a clear <code>console.log</code> .
One edge case to remember?	If the value has a <code>toJSON</code> method, the snapshot clones what <code>toJSON</code> returns instead of the raw object.

◆ **Source:** `02-runes/02-$state.md` (`$state.snapshot`)

• • •

Q9 — Synchronous feedback: a `$state` change can lag a beat behind a click. How do I force the UI to reflect it instantly?

⚡ **Short answer.** Wrap the value in `$state.eager(value)`. It reflects the change on screen the moment it happens, skipping Svelte's usual update coordination. Use it sparingly — only for instant feedback to a user action.

🧠 **Theory.** Normally Svelte is clever about *when* it repaints: it batches changes together and coordinates them for smoothness, and if a value happens to be waiting on something slow (an `await`), the visible update can lag a beat. Almost always that's what you want. But occasionally you want an *instant* visual reaction to a click — the classic case being a navigation bar, where you want the link the reader just pressed to light up right away, while the next page is still loading, so they feel the click land. `$state.eager(value)` does exactly that: it reflects the change on screen the moment it happens, skipping the usual coordination. It's a specialist tool — use it sparingly, only to give feedback in response to a user action, and let Svelte time everything else.

Component map.

- `SectionNav.svelte` → gets `pathname` (prop) — `eager` so the active link flips at once

📁 The files.

📄 `SectionNav.svelte`

```
<script>
  let { pathname } = $props();
</script>

<nav>
  <a href="/world" aria-current={$state.eager(pathname) === '/world'
? 'page' : null}>World</a>
  <a href="/sports" aria-current={$state.eager(pathname) ===
'/sports' ? 'page' : null}>Sports</a>
</nav>
```

🔑 **The move.** `$state.eager(value)` for instant visual feedback on a user action — and nowhere else.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Why might a <code>\$state</code> change not show on screen the instant it happens?	Svelte batches and coordinates updates for smoothness, and a value waiting behind an <code>await</code> can lag a beat.
How do you force an immediate on-screen update?	Wrap the value in <code>\$state.eager(value)</code> , which reflects the change at once, skipping the usual coordination.
When is <code>\$state.eager</code> appropriate?	Only for instant visual feedback to a user action — like lighting up the nav link they just clicked while the next page loads.
What's the rule on using it?	Use it sparingly; let Svelte time every other update, since coordinated updates generally give a better experience.

◆ **Source:** `02-runes/02-$state.md` (`$state.eager`; hrefs adapted to our sections)

• • •

Passing state around

Q10 — Passing state to a function: I call a function with a state value (`greet(name)`). Does it get a live link to `name`, or just a copy of its current value?

⚡ **Short answer. Just a copy, frozen at call time.** JavaScript passes values, not variables, so if you *store* the result it won't update when `name` changes. Inside a component you avoid this by not storing it — call the function in the markup, or make it a `$derived`. To hand the live value across a boundary, pass a getter, `() => name`.

🧠 **Theory.** You know how a function takes an argument: `greet(who)` receives whatever you pass for `who`. Now call it with component state: `greet(name)`. JavaScript is *pass-by-value* — it copies the current value of `name` into `who` at the moment of the call. The function gets the string `'Ada'`, not the variable `name`. So if you store the result in a plain variable, that variable is computed once and frozen; changing `name` afterwards does nothing to it.

Here's the reassuring part for everyday components: when you call `greet(name)` directly in your markup, Svelte re-runs the markup whenever `name` changes, so it re-reads the current `name` and calls `greet` again — the greeting stays fresh on its own. You only hit the frozen-copy bug when you *store* the result. And the right tool for "a value computed from state that should stay fresh" is `$derived` (Q14, lecture 5).

There is one case where you genuinely need to hand the *live* value into a function: when the function lives across a boundary — a reusable helper, a `.svelte.ts` module, a class — and will read it again later. Then you don't pass the value, you pass a tiny getter, `() => name`, and the function calls it to read the latest each time. That's exactly the `getReader()` getter in Q12.

Component map.

- `Greeting.svelte` → owns `name` (`$state`)
 - `let message = greet(name)` → ran once → frozen at "Hello, Ada"
 - `{greet(name)}` in markup → re-runs on every change → always fresh

The files.

Greeting.svelte

```
<script>
  let name = $state('Ada');

  function greet(who) {
    return `Hello, ${who}`;
  }

  let message = greet(name); // runs once with 'Ada', then frozen
</script>

<button onclick={() => name = 'Grace'}>rename</button>

<p>{message}</p>      <!-- ✖ stays "Hello, Ada" — computed once -->
<p>{greet(name)}</p> <!-- ✔ "Hello, Grace" — re-runs on every change -->
>
```

🔑 **The move.** Passing `$state` into a function hands over a frozen copy. Don't store the result — call it inline or make it `$derived`; to keep it live across a boundary, pass a getter, `() => name`.

🔧 **Going further.** This is the same fact that makes `$derived` necessary (a stored computation goes stale) and that makes the cross-module getter in Q12 work (`getReader()` hands back the live value on demand). "Pass a function, not a value" also covers getter properties and a proxy's get/set — anything that re-reads when it's called.

🔄 **Quick-fire review.**

QUESTION	ANSWER
When you pass a <code>\$state</code> value into a function, does it get a live link or a copy?	A copy of the value as it stands at that moment — JavaScript passes values, not variables.
If you store the result (<code>let m = greet(name)</code>), does it update when <code>name</code> changes?	No — it was computed once and is frozen; later changes to <code>name</code> don't reach it.
Then why does <code>{greet(name)}</code> in the markup stay fresh?	Because Svelte re-runs the markup on every change, so it re-reads <code>name</code> and calls <code>greet</code> again.
How do you hand the live value to a function across a boundary?	Pass a getter — a small function like <code>() => name</code> — that the function calls to re-read the latest.

◆ **Source:** reconstructed from `02-runes/02-$state.md` (passing state into functions)

• • •

Sharing state across files

Q11 — Sharing across files: where do I put one value — the logged-in reader — that many components read and update together?

⚡ **Short answer.** Put it in a `.svelte.ts` module — an ordinary file where runes are allowed — and export an **object** holding the value. Every component that imports it shares the same live copy; change a property (`session.reader = person`) and they all update.

🧠 **Theory.** Most state lives inside one component: `let claps = $state(0)` belongs to that one card. But some values belong to no single component — who's logged in, the light-or-dark theme, the items in a cart. Components scattered all over the app need the *same* one, and threading it down through props to every corner would be madness. So you lift it out of the components and put it in a *module*: just a separate file you import things from.

There's one catch. Runes like `$state` normally only work where Svelte compiles them, which is inside components. To use a rune in a standalone file, you give the file a `.svelte.ts` ending (or `.svelte.js`). That suffix tells the compiler "treat this like component code — runes allowed here," so now you can declare `$state` in it and export it. The one rule is *how* you export it: export an **object** and change its properties. Write `export const session = $state({ reader: null })`, and to log someone in, do `session.reader = person`. Every component that imports `session` holds the very same object, so the instant you set `session.reader`, they all update. What you must not do is export a bare variable and then *reassign* it — that one form quietly loses its reactivity across files (see the gotcha). Mutating a property on a shared object avoids the whole issue.

Component map.

```

▸ session.svelte.ts → owns session ($state object) – the logged-in reader
  | imported by ■ Masthead.svelte → reads session.reader (greeting)
  | imported by ■ CommentForm.svelte → reads session.reader (byline)

```

📁 The files.

📄 session.svelte.ts

```

export const session = $state({ reader: null });

export function logIn(person) {
    session.reader = person;    // change a property, never reassign
    `session`
}

```

📄 Masthead.svelte

```
<script>
  import { session } from './session.svelte.ts';
</script>

{#if session.reader}Hi, {session.reader.name}{/if}
```

📄 CommentForm.svelte

```
<script>
  import { session } from './session.svelte.ts';
</script>

<small>posting as {session.reader?.name ?? 'guest'}</small>
```

🔑 **The move.** Share app-wide state from a `.svelte.ts` module by exporting an object you mutate — never a bare variable you reassign.

⚠️ **Gotcha.** The one form that breaks: `export let reader = $state(null)` and then reassigning `reader = person`. A reassigned export loses its reactivity once another file imports it. Always change a property on a shared object (above), or go through functions (Q12).

🔧 **Going further.** Why does reassigning break but mutating a property work? Svelte compiles each file on its own, turning every state read and write into get/set calls on a hidden "signal." The importing file never gets that rewrite for a reassigned export, so it receives the raw signal object — `typeof === 'object'`, not your value. A shared object dodges this because every file holds the same reference.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Where do you put state that many components across the app must share?	In a <code>.svelte.ts</code> module — an ordinary file where runes are allowed — and import it wherever you need it.
Why must the file end in <code>.svelte.ts</code> rather than <code>.ts</code> ?	Runes like <code>\$state</code> only work where Svelte compiles them; the <code>.svelte.ts</code> suffix tells the compiler to treat the file like component code.
What's the correct way to export shared state?	Export an object and change its properties (<code>session.reader = person</code>); every importer shares the same reactive object.
What must you never do?	Export a bare variable and then reassign it — a reassigned export loses its reactivity once another file imports it.

◆ **Source:** `02-runes/02-$state.md` (passing state across modules), reconstructed as `session`

• • •

Q12 — Sharing, encapsulated: how do I let components read the logged-in reader but route every change through one controlled place?

⚡ **Short answer.** Keep the value *private* inside the `.svelte.ts` module — don't export it — and export **functions** instead: a getter to read it, and actions like `login` / `logout` to change it. Components call the functions; the value itself never leaves the file.

🧠 **Theory.** Q11's exported object works, but it's wide open: any component can write `session.reader = anything`, with no guard rail. Often you want the opposite — a single, controlled doorway, so every change runs through one place where you could validate it, log it, or call the server. That's *encapsulation*: hide the data, expose only controlled ways to touch it.

You get it by keeping the state as a private variable inside the module — declared with `$state`, but *not exported* — and exporting functions instead: a `getReader()` to read the current value, and `login(person)` / `logout()` to change it. Because the variable never leaves the file, every read and write to it happens inside that one module, which also sidesteps the reassignment trap from Q11 entirely (outside files only ever call functions; they never touch the variable). And it stays reactive: when a component calls `getReader()` in its markup, it reads the `$state` inside, so that spot re-runs whenever the reader changes.

Component map.

```
▸ session.svelte.ts → owns reader ($state — PRIVATE, never exported)
  ● exports getReader(), login(), logout() → the only doorway in
    ↳ imported by ■ Masthead.svelte → calls getReader() to read, login() to
      change
```

📁 **The files.**

 session.svelte.ts

```
let reader = $state(null); // private – not exported

export function getReader() {
  return reader;
}

export function logIn(person) {
  reader = person;
}

export function logOut() {
  reader = null;
}
```

 Masthead.svelte

```
<script>
  import { getReader, logIn } from './session.svelte.ts';
</script>

{#if getReader()}
  Hi, {getReader().name}
{:else}
  <button onclick={() => logIn({ name: 'Ada' })}>log in</button>
{/if}
```

 **The move.** To guard or hide shared state, keep it private in the module and expose it only through functions.

 **Going further.** Object (Q11) or functions (Q12): reach for the object when you just want shared data, for functions when changes need a guard rail. When it grows, a *reactive class* — a `Session` class with `$state` fields and `logIn` / `logOut` methods (see Q4) — packages the private state and its actions together, the cleanest version of this pattern.

 **Quick-fire review.**

QUESTION	ANSWER
How do you share state but stop components from changing it directly?	Keep the state private in the module (don't export it) and export functions — a getter plus actions — as the only way in.
Why route changes through functions instead of exposing the object?	So every change runs through one controlled place where you can validate it, log it, or call the server.
Is the value still reactive if components read it through <code>getReader()</code> ?	Yes — calling the getter in markup reads the <code>\$state</code> inside, so that spot re-renders whenever it changes.
What's the cleanest version when this grows?	A reactive class — a <code>Session</code> class with <code>\$state</code> fields and <code>logIn</code> / <code>logOut</code> methods — packaging the private state with its actions.

◆ Source: `02-runes/02-$state.md` (the private-state-plus-functions pattern)

• • •

Going further

Q13 — One file, two components: can I define both `ArticleCard` and `ClapButton` in a single `.svelte` file?

⚡ **Short answer. No.** A `.svelte` file is exactly one component. Show two components as two files. (Snippets reuse markup *inside* one component; `<svelte:self>` only recurses — neither is a second component.)

🧠 **Theory.** A quick but useful boundary. In Svelte, a `.svelte` file *is* one component — one box. There is simply no syntax to declare two separate components inside a single file. So when you want to show the clap button talking to the article card, the honest answer is two files: `ArticleCard.svelte` and `ClapButton.svelte`, with the card importing and placing the button (that parent-child conversation is the props-and-events topic, lecture 7). Two things sometimes get mistaken for a workaround. *Snippets* let you reuse a chunk of markup inside one component, but a snippet isn't a separate component — it has no `<script>` or state of its own, it just shares the host's. And `<svelte:self>` only lets a component render *itself* again, for recursion like a comment showing its replies. Neither gives you two real components in one file, because that thing doesn't exist.

Component map.

- `ArticleCard.svelte` → the parent (one file)
 - └ ■ `ClapButton.svelte` → a child component (a separate file)
 - × the two cannot share one `.svelte` file

📁 The files.

📄 `ArticleCard.svelte`

```
<script>
  import ClapButton from './ClapButton.svelte';
</script>

<article>
  <h2>Ceasefire Talks Collapse</h2>
  <ClapButton />
</article>
```

📄 `ClapButton.svelte`

```
<button>clap</button>
```

🔑 **The move.** One file, one component. Relationships are always multiple files plus a map.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Can two separate components live in one <code>.svelte</code> file?	No — a <code>.svelte</code> file is exactly one component; there's no syntax for two.
How do you show two components and their relationship?	Write two files (e.g. <code>ArticleCard.svelte</code> and <code>ClapButton.svelte</code>) plus a map, with one importing the other.
Aren't snippets a way to get a second component into one file?	No — a snippet reuses markup inside one component; it has no <code><script></code> or state of its own.
What does <code><svelte:self></code> do, then?	It lets a component render itself again, for recursion (like a comment showing its replies) — not a second component.

◆ **Source:** reconstructed (the Svelte component model)

• • •

Q14 — \$state vs \$derived vs \$effect: which one holds the "5 min read" label computed from the article body?

⚡ **Short answer. None of them holds it.** A value computed from other values is a `$derived` (lecture 5). `$state` is for values you *hold and change*; `$effect` is for *doing* something in the world. Computing the label by hand into `$state` would go stale.

🧠 **Theory.** Three runes sit near each other, and picking the wrong one is a common mistake, so here's the deciding question. `$state` is for a value you *hold and change over time* — the clap count, a menu's open-or-shut flag, the text in a box. But not every value is held. The "5 min read" label isn't stored anywhere; it's *computed* from the article's body length. A value computed from other values is the job of `$derived` (lecture five), which recalculates itself whenever its ingredients change — never `$state`, and never an `$effect` (an effect is for *doing* something out in the world, like saving to a server). So the deciding question, asked of any piece of behaviour: am I *holding* a value (state), *computing* one from other values (derived), or *doing* something in the world (effect)? If you tried to compute the read-time once by hand into a `$state` variable, it would go stale the moment the body changed — which is precisely the bug `$derived` exists to kill.

Component map.

- ArticleCard.svelte
 - body (`$state`) → a value you hold
 - readMins (`$derived`) → computed from body (lecture 5), NOT `$state`

📁 The files.

📄 ArticleCard.svelte

```
<script>
  let body = $state('...the full article text...');

  // computed from body, so it's $derived – not $state, not an
  $effect:
    // let readMins = $derived(Math.ceil(body.split(' ').length /
    200)); ← lecture 5
</script>
```

🔑 **The move.** Hold a value → `$state`. Compute a value → `$derived`. Touch the world → `$effect`.

🔄 **Quick-fire review.**

QUESTION	ANSWER
Should the "5 min read" label, computed from the body, be <code>\$state</code> ?	No — a value computed from other values belongs in <code>\$derived</code> , which recomputes itself when its inputs change.
What is <code>\$state</code> for?	A value you hold and change over time — a clap count, an open/closed flag, the text in a box.
What is <code>\$effect</code> for?	Doing something out in the world when state changes — saving to a server, drawing to a canvas — not producing a value.
What goes wrong if you compute a value into <code>\$state</code> by hand?	It goes stale the moment its inputs change, which is exactly the bug <code>\$derived</code> exists to prevent.

◆ **Source:** `claude-discussions` (the deciding question); deepened in lecture 5

• • •

Q15 — Framework bridge: is `$state` just React's `useState` or Vue's `ref` under a new name?

⚡ **Short answer. Almost.** `$state` is Vue's `ref` without the `.value`, and React's `useState` without the setter and the never-mutate rule. The deep difference shows in `.svelte.ts`: that factory runs once, where a React hook re-runs every render.

🧠 **Theory.** If React and Vue are a faint memory, here's the bridge. In React you made a reactive value with `useState`, which hands you the value plus a setter function, with the iron rule that you must never change the value directly — always call the setter. In Vue you used `ref`, which wraps the value in a little box you reach through `.value` on every read and write. `$state` is the same idea with the ceremony stripped out: it's Vue's `ref` *without* the `.value` (Svelte's compiler reaches into the box for you), and React's `useState` *without* the setter and the never-mutate rule (you just change the variable). The deeper difference surfaces later, when you move reactive logic into a `.svelte.ts` file: that factory function runs *once* and the reactivity lives inside the value, whereas a React hook re-runs top to bottom on every single render. But for the everyday clap count, the takeaway is short: it's a plain variable, and the compiler does the wiring the other two made you do by hand.

Component map.

the same reactive count, three dialects:

- React `setCount(c + 1)` — a setter
- Vue `count.value++` — the `.value` tax
- Svelte `count++` — just a variable

📁 The files.

📄 Counter.jsx (REACT, for comparison)

```
const [count, setCount] = useState(0);
setCount(count + 1); // a setter, and never `count++`
```

📄 Counter.vue (VUE, for comparison)

```
const count = ref(0);
count.value++; // the `.value` tax, every read and write
```

App.svelte (SVELTE)

```
<script>
  let count = $state(0);
</script>

<button onclick={() => count++}>{count}</button>
```

🔑 **The move.** `$state` = `ref` minus `.value` = `useState` minus the setter. Just a variable, and the compiler does the wiring.

🔄 **Quick-fire review.**

QUESTION	ANSWER
How does <code>\$state</code> relate to Vue's <code>ref</code> ?	It's <code>ref</code> without the <code>.value</code> — the compiler reads the box for you, so you use the variable directly.
How does it relate to React's <code>useState</code> ?	It's <code>useState</code> without the setter and the never-mutate rule — you just change the variable.
What's the deep difference from a React hook?	A <code>.svelte.ts</code> factory runs once and the reactivity lives in the value; a React hook re-runs top to bottom on every render.
One-line summary?	<code>\$state</code> is a plain variable the compiler wires up — doing by hand what React and Vue made you do explicitly.

◆ **Source:** for comparison (React, Vue); framing from `claude-discussions`